

Bharat MCP — Bangalore Pilot: Infrastructure Plan

500-restaurant pilot · infra lead + 2 interns · target 12-14 weeks

1. Design Principle

One shared, config-driven MCP server serving all 500 restaurants — not 500 individual deployments. Every restaurant is a row of data (menu, hours, pricing, contact) inside one multi-tenant system. This is the single decision that keeps the pilot buildable by one engineer: the interns’ job is to fill the data model, not to touch code, and adding restaurant #501 later costs nothing in engineering time.

2. System Architecture

Components:

Layer	Purpose	Suggested choice
MCP protocol server	Exposes tools to AI agents (Claude, ChatGPT, Gemini)	TypeScript/Node using the official MCP SDK, or Python (FastMCP) — pick whichever your lead is faster in
Application/business logic	Order validation, availability checks, restaurant lookup, rate limiting	Same runtime as MCP server, thin service layer
Primary database	Restaurants, menu items, categories, hours, orders, bookings	Postgres (managed — RDS / Supabase / Neon)
Cache/session store	Fast lookups, idempotency keys, rate-limit counters	Redis (managed — Upstash is cheap at this scale)
Object storage	Menu photos, item images	S3-compatible (Cloudflare R2 is cheaper than S3, no egress fees)
Order relay	Push confirmed orders to restaurants that have no POS	WhatsApp Business API (or plain SMS/voice call fallback)
Payments	UPI collection for prepaid orders	Razorpay or Cashfree UPI intent/deep-link API
Admin/onboarding tool	Lets interns add & edit restaurants without touching code	Lightweight internal web form, or a validated Google Sheet + sync script for the very first weeks
Monitoring	Uptime, errors, logs	Sentry (errors) + UptimeRobot/Better Uptime (uptime) + basic structured logging to a hosted log sink

Flow: AI agent → MCP server (tool calls: search, get_menu, check_availability, place_order, get_order_status, book_table) → application layer validates against Postgres/Redis → order confirmed → relayed to restaurant via WhatsApp/SMS → restaurant accepts/rejects → status pushed back through MCP to the agent → payment settled via UPI intent or marked COD.

3. Data Model (minimum viable schema)

- restaurants** — id, name, address, geo-coordinates, phone, WhatsApp number, cuisine tags, hours, status (active/paused/pending), owner consent flag, onboarding date
- menu_categories** — id, restaurant_id, name, sort order
- menu_items** — id, category_id, name, description, price, veg/non-veg flag, image_url, availability flag, spice/allergen tags (optional for pilot)
- availability_overrides** — id, restaurant_id, date/time range, reason (e.g., sold out, closed for event)
- orders** — id, restaurant_id, agent/platform source, items, total, payment mode, payment status, order status (placed/accepted/rejected/fulfilled/cancelled), timestamps
- bookings** (if table reservations are in scope) — id, restaurant_id, party size, time slot, status

Menu data enters through the admin tool as structured fields, not free-text — this is what makes it usable by an AI agent at all; a photographed menu still needs to be typed into this schema by the interns.

4. MCP Tool Surface (what the AI agents actually call)

- `search_restaurants(location, cuisine, query)` — returns matching restaurants within the pilot’s Bangalore coverage
- `get_menu(restaurant_id)` — structured categories + items + prices + availability
- `check_availability(restaurant_id, item_ids / party_size, time)` — real-time stock/table check
- `place_order(restaurant_id, items, customer_contact, payment_mode)` — creates order, triggers relay to restaurant
- `get_order_status(order_id)` — polling/webhook-style status for the agent to relay to the end user
- `book_table(restaurant_id, party_size, time)` — optional for pilot scope, add only if Dineout-style booking is part of what you’re testing

Each tool needs input validation, clear error messages (agents need to explain failures to end users), and idempotency keys on `place_order` so a retried agent call doesn’t double-book.

5. Onboarding Pipeline (infra side of what the interns do)

1. Intern collects menu photos/PDF + hours + owner consent (WhatsApp-first).
2. Data entered into the admin tool → validated (required fields, price sanity checks, duplicate detection) → written to Postgres.
3. Automated smoke test: a scripted call against every new restaurant’s `get_menu` and `place_order` (test mode, no real charge) before it goes live.
4. Restaurant flipped from `pending` to `active`.

Building the admin tool and the smoke-test script is ~1-1.5 weeks of the lead’s time in week 1-2 and is what actually determines whether interns can scale onboarding without waiting on engineering.

6. AI Platform Integration

- Register the MCP server per platform’s connector process (Anthropic, OpenAI, Google each have their own registration/auth flow — expect API-key or OAuth-style auth, not identical across platforms).
- Run structured test transcripts on each platform: search → menu → order → status, checking that error states (item unavailable, restaurant closed, payment failure) surface as sensible agent-facing messages, not raw stack traces.
- Expect iteration here — this is the part most likely to eat unplanned lead time, since each platform’s MCP client behavior differs slightly.

7. Hosting & Scaling for Pilot Scale

500 restaurants with low pilot order volume is a small workload — this does not need a large cluster.

- Start on a single managed compute instance (e.g., a small ECS Fargate service, or even a single well-monitored VPS) behind a load balancer, with Postgres and Redis as managed services rather than self-hosted.
- Horizontal scaling only becomes a real conversation past a few thousand restaurants or meaningful order volume — don’t over-build this for the pilot.
- Keep infra-as-code (even a simple Terraform or Docker Compose setup) from day one so the same environment can be redeployed if you move providers after the pilot.

8. Security & Compliance

- API keys per AI platform integration; rate-limit all MCP endpoints.
- HTTPS everywhere; encrypt customer contact info at rest.
- Restaurant owner consent recorded (a simple signed/logged opt-in) before any restaurant is listed — this matters if a restaurant later disputes being listed.
- Payment flow via a licensed PSP (Razorpay/Cashfree) — never handle card/UPI credentials directly.
- Basic RBAC on the admin tool so interns can only edit restaurants, not touch infra config.

9. Monitoring & QA

- Sentry for error tracking on the MCP server and application layer.
- Uptime monitoring on the MCP endpoint itself (agents will silently fail over or error out to end users if it’s down — you want to know before they do).
- A weekly manual QA pass: pick 10 random live restaurants, run a real order through an actual AI agent session, confirm it reaches the restaurant and status updates correctly.

10. Team Split (infra-relevant)

- **Lead (infra):** weeks 1-2 build MCP server + schema + admin tool; weeks 2-4 platform integration + test harness; weeks 4-12 mostly QA, edge cases, monitoring (~30-40% time), remainder free for other work (e.g., partner conversations with POS vendors).
- **Interns:** not infra-facing — they operate through the admin tool once built; no code access needed.

11. Infra Cost Estimate (pilot, 3 months)

Item	Rough monthly cost	3-month total
Compute (small managed instance/service)	₹5k-10k	₹15k-30k
Managed Postgres	₹3k-8k	₹9k-24k
Managed Redis	₹1k-3k	₹3k-9k
Object storage (images)	₹500-1.5k	₹1.5k-4.5k
WhatsApp Business API	₹2k-5k (usage-based)	₹6k-15k
Payment gateway (UPI) fees	Transaction-based, negligible at pilot order volume	—
Error/uptime monitoring (Sentry + UptimeRobot, free/low tiers at this scale)	₹0-2k	₹0-6k
LLM/agent testing credits	₹2k-5k	₹6k-15k
Total infra	—	≈ ₹40k-100k over 3 months

This sits on top of (not instead of) the ₹90k-150k in intern stipends and any optional restaurant onboarding incentive already estimated for the ops side. All-in pilot cost including infra is roughly ₹2.5-4.5L (~\$3,000-5,500).

12. Key Infra Risks

- **AI platform integration friction** — each platform’s MCP client/auth quirks are the most likely source of schedule slip; budget slack in weeks 2-4.
- **No-POS restaurants** — most long-tail restaurants have no order-management system; the WhatsApp/SMS relay is a manual bridge, not real-time confirmation, so agent-facing order status will sometimes lag. Set expectations with restaurants accordingly.
- **Data staleness** — menu/price changes at a physical restaurant won’t reach the system unless there’s a simple update path back to the admin tool; without this, agents will serve stale prices.
- **Single point of failure** — one shared MCP server means an outage affects all 500 restaurants at once; monitoring and a fast rollback path matter more than scale here.

13. What This Doc Deliberately Leaves Out

Restaurant sourcing/outreach, sales conversion, and onboarding incentives are ops-side, covered separately — this is the infra/engineering spec only.